

TECHNICAL DEEP DIVE

解构 vLLM KV Connector: 从调度解耦到全局零拷贝池化

基于技术演讲与配套材料整理

配套材料: vllm-kv-connector-mini-lesson - yihua 2.pptx、vllm-mini-course.pptx

2026-08-03

当大模型推理从单轮问答走向多轮 Agent 交互，KV Cache 的规模随对话轮次持续膨胀，单机显存很快捉襟见肘。将 KV Cache 搬出引擎、跨节点共享、按需加载 - 这件事听起来直觉明了，工程上却要求推理引擎在调度、传输和存储三个维度同时给出干净的抽象。vLLM 的 KV Connector 正是为此而生的接口层。本文沿着它从 v0 到 v1 的架构演进，逐层拆解异步传输的三种范式，最终落脚到 LMCache 与 Mooncake 两个生态系统如何借助 CudaIPC 和 GPUDirect RDMA 实现零拷贝全局池化。

适读人群： 具有一定大模型推理基础，希望深入了解 vLLM 底层架构、KV Cache 优化机制及分布式存储系统集成的后端工程师与系统架构师。

前置知识：

- **KV Cache** (Key-Value Cache)：大模型推理过程中缓存的键值对，用于避免对已处理 token 的重复计算。
- **Prefill-Decode (PD)**：大模型推理的两个阶段。Prefill 一次性处理全部输入 token，计算密集；Decode 逐 token 生成，访存密集。
- **vLLM**：高性能大语言模型推理引擎，本文讨论的核心系统。
- **chunked prefill** (分块预填充)：将一个请求的 prefill 拆分成多个 step 执行的机制，在 vLLM v1 中成为默认配置。
- **LMCache**：独立的 KV Cache 管理层，采用守护进程模式与推理引擎协作。
- **Mooncake**：以 KV Cache 为中心的分布式存储系统，支持 PD 分离与全局 KV Cache 池化。

阅读目标：

- 1 理解大模型推理中 KV Cache 外部化的工程背景与演进动力。
- 2 掌握 vLLM v1 KV Connector 的非侵入式注入架构及调度解耦原理。
- 3 深入剖析隐藏 I/O 延迟的三种异步传输范式：逐层传输、请求级异步与 L2 预取。
- 4 了解 LMCache 与 Mooncake 如何利用 CudaIPC 和 RDMA 实现零拷贝全局池化。

一、长上下文时代的 KV Cache 危机与早期探索

本节要回答的问题： 为什么大模型推理需要将 KV Cache 搬到引擎外部？vLLM v0 中的早期方案为何在架构演进后走进死胡同？

1.1 多轮对话下的 KV Cache 膨胀

在传统单轮问答场景中，prefill 的开销是一次性的。然而，当业务形态转向 **Agentic Workloads** - 模型在多轮交互中持续调用工具、检索上下文、再次推理 - 每一轮都会在已有 prompt 上追加新内容，累积的 prefill token 数量随轮次显著增长。如果每轮都从头计算 KV Cache，prefill 延迟将迅速成为端到端响应的瓶颈。

核心问题随之浮出水面：**能否把上一轮甚至跨请求计算好的 KV Cache 保留下来，直接复用？**

引擎内部的前缀缓存（prefix caching）只能覆盖同一实例、同一进程内的场景。一旦涉及 PD 分离部署或跨节点调度，KV Cache 就必须被"搬出去" - 写入外部存储、网络传输到另一台机器、再加载回 GPU。这正是外部 KV Cache 需求的本质来源。

1.2 CacheGen 的启发与 v0 的尝试

学术界对这一方向的早期代表是 **CacheGen**（发表于 SIGCOMM 2024），它提出了基于 chunk 的 KV Cache 压缩与外部存储机制。这项工作最初在 HuggingFace Transformers 上实现原型，为了接入真实的高性能推理引擎，研究者将其集成进 vLLM - 这也是 vLLM 开发 KV Connector 的最初动机。

在 vLLM v0 版本中，整个引擎运行在单进程内，每个请求执行完整的逐请求 prefill。在这种相对简单的架构下，KV Connector 的实现策略是：**魔改 attention metadata，欺骗 Worker 使其认为相关 token 已命中缓存，从而跳过重复的 prefill 计算。**好处是对核心代码侵入极小 - 不需要改调度器或模型前向逻辑，只需在 attention 层的元数据上动手脚。

但隐患同样明显：这种"伪装 cache hit"的技巧与 attention 后端的内部数据结构高度耦合。每换一种后端，metadata 格式都不同，维护成本随后端种类线性增长。

1.3 架构大迁移：三座大山

vLLM 从 v0 演进到 v1 时，底层架构发生了根本性变化，旧的 Connector 方案随之失效。下图概括了这次迁移带来的三个核心挑战：

vLLM v0 → v1

架构大迁移：三座大山



图注：vLLM v0→v1 架构迁移中 KV Connector 面临的三个关键挑战。来源：演讲 PPT 第 6 页

图中左侧展示了架构演进方向：v0 的"单进程 · 逐请求 prefill"变为 v1 的"进程分离 · chunked prefill"。右侧三个挑战逐一说明如下：

挑战	v0 状态	v1 变化	对旧 Connector 的冲击
Attention metadata 维护	单一后端，魔改可控	MLA 等复杂 attention 出现，每种 metadata 结构不同	伪装 cache hit 越来越不可持续
Scheduler 与 Worker 分离	同一进程直接通信	调度和执行拆到不同进程	需要显式跨进程透传 Connector 状态，旧方案无此通道
chunked prefill 成为默认	一次性完成 prefill	prefill 被拆成多个 step	KV Cache 加载与保存时机碎片化，不再有单一触发点

1.4 因果链：旧方案为何不可持续

将上述因素串联：Agentic Workloads 使 KV Cache 复用成为刚需 → v0 选择最小侵入的 metadata 魔改方式 → v1 引入进程分离，跨进程传递 Connector 状态缺乏机制 → chunked prefill 成为默认，KV Cache 分步加载让旧方案无法感知中间状态 → MLA 等新型 attention 机制的涌现使按后端魔改的维护成本急剧上升。五个因素叠加，侵入式设计彻底走向死胡同。

核心教训： v0 方案把 Connector 逻辑嵌入了 attention 执行的细节中，而非建立在调度与执行之间的稳定抽象上。当底层架构剧烈变化时，这种耦合必然断裂。

边界说明： 演讲材料未给出 v0 Connector 的具体性能数据或所支持的 attention 后端列表，上述分析基于架构层面的定性推演。

二、解耦调度与执行：v1 KV Connector 的注入式设计

本节要回答的问题： 在 Scheduler 与 Worker 运行于不同进程的 vLLM v1 架构下，"哪些 token 可以复用"的决策与"数据搬进搬出"的执行如何在进程边界上干净地分开？

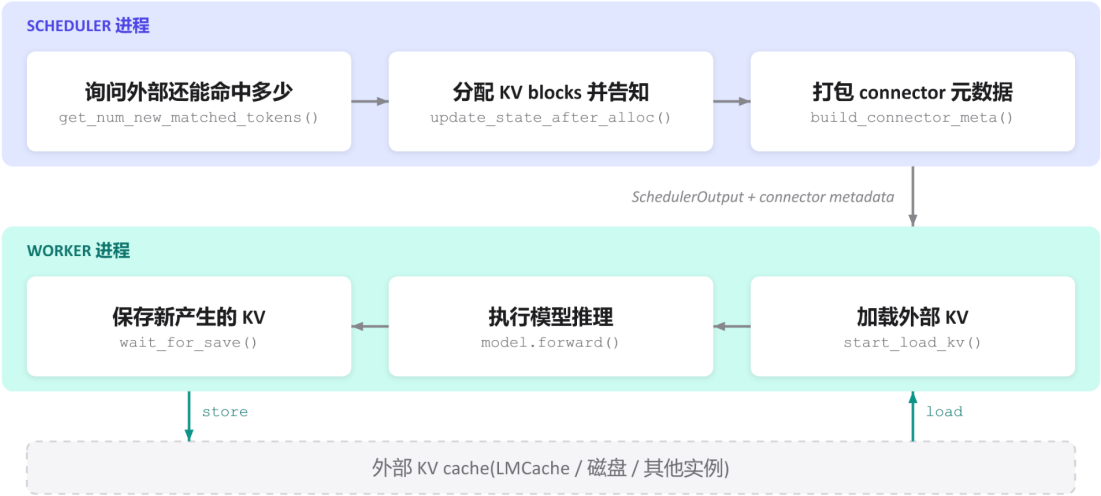
vLLM v1 的答案是**注入而非侵入** - 在原生调度和执行流程的固定阶段插入接口调用，而不修改 Scheduler 或 Worker 已有的核心逻辑。

2.1 端到端工作流全景

下图展示了一次完整请求经过 Scheduler、KV Cache 和 Worker 的全生命周期。理解这张图是把握后续所有异步优化的前提，后续范式本质上都是在这条基线流程上做时间维度的重叠。

VLLM V1 · END-TO-END WORKFLOW

端到端 workflow



图注：vLLM v1 KV Connector 端到端 workflow。上半部分为 Scheduler 进程，下半部分为 Worker 进程，底部为外部 KV Cache 存储。来源：演讲 PPT 第 8 页

图中自上而下包含以下关键元素与流转：

序号	所在进程	操作	调用接口	作用
1)	Scheduler	询问外部命中数量	<code>get_num_new_matched_tokens()</code>	查询当前请求的 prompt 中有多少 token 的 KV 已被外部缓存
2)	Scheduler	分配 KV blocks	<code>update_state_after_alloc()</code>	决定哪些 block 从外部加载、哪些本地计算，并完成显存 block 分配
3)	Scheduler	打包元数据	<code>build_connector_meta()</code>	将命中信息、block 映射打包进 connector metadata，附加到 SchedulerOutput
4)	-	跨进程消息	SchedulerOutput + connector metadata	序列化后发送给 Worker
⑤	Worker	加载外部 KV	<code>start_load_kv()</code>	根据元数据从外部存储读入对应 block
⑥	Worker	模型推理	<code>model.forward()</code>	正常前向计算，对 Connector 透明
⑦	Worker	保存新 KV	<code>wait_for_save()</code>	将本轮新计算出的 KV 写回外部存储

图中底部用双向箭头标注了 **load** 和 **store** 两条数据通路，均发生在 Worker 与外部存储之间。Scheduler 全程不触碰实际的 KV 数据。

2.2 Scheduler 侧：只做决策，不碰数据

Scheduler 在每个调度 step 中依次完成三件事：

- 1 **查命中** - 调用 `get_num_new_matched_tokens()`，拿到一个整数，表示外部存储能提供多少 token 的 KV。命中越多，需要实际计算的就越少。
- 2 **分 block** - 调用 `update_state_after_alloc()`，将命中 token 范围映射到物理 block 编号上。与 vLLM 原生 block 分配逻辑衔接，只是多了"这些 block 的内容将由外部填充"的标记。
- 3 **打包元数据** - 调用 `build_connector_meta()` 序列化。元数据体积极小（block 编号列表、命中长度等），不含任何 KV 张量。

三步完成后，connector metadata 附在 `SchedulerOutput` 上一并发出。对 Scheduler 的其余逻辑而言，这三个调用是**纯粹的增量注入** - 不改变请求排序、抢占策略或 chunked prefill 的分片决策。

2.3 Worker 侧：只管搬运，不做决策

Worker 收到消息后按固定顺序执行：**load** → **forward** → **store**。 `start_load_kv()` 读取元数据中标记的 block 列表，向外部存储发起搬运，目的地是 Scheduler 已分配好的本地显存 block；`model.forward()` 正常执行前向推理；`wait_for_save()` 将新产生的 KV 写回外部。

整条链路的关键特性是：**Worker**

不持有任何"该不该命中"的判断逻辑，它看到的只是"从哪搬、搬到哪、搬多少"的指令集。

2.4 最小状态演进示例

假设一条请求的 prompt 包含 8 个 token，外部已缓存前 6 个 token 的 KV：

阶段	位置	动作	结果
查命中	Scheduler	<code>get_num_new_matched_tokens()</code> 返回 6	得知前 6 token 可外部获取
分 block	Scheduler	为 8 token 分配 block，前 6 标记"外部填充"	映射写入元数据
load	Worker	搬入 6 token 的 KV	显存对应 block 就绪
forward	Worker	仅对第 7、8 token 执行 prefill	KV Cache 完整覆盖 8 token
store	Worker	新计算的 2 token KV 写回外部	外部缓存更新为 8 token

实际计算量从 8 token 降至 2 token，收益直接取决于外部命中率。

2.5 设计因果链与局限

- **Scheduler 掌握全局视图**（请求队列、block 空闲表、外部命中信息），由它做"复用还是计算"的决策。
- **Worker 掌握硬件通路**（GPU 显存、PCIe / RDMA 链路），由它执行实际搬运。
- **connector metadata 是唯一的跨进程契约**：体积小、语义明确、不含张量负载。

新的外部存储后端只要实现上述六个接口，即可"插入"原有流程，Scheduler 无需关心 KV Cache 的物理位置。

但值得注意的是，上述流程是**同步语义**的：load 在 forward 之前，store 在 forward 之后。这意味着 load 和 store 期间 GPU 处于等待状态。当外部存储延迟较高时，这段空转将成为吞吐瓶颈。要消除这一等待，就必须让数据搬运与 GPU 计算在时间上重叠。

三、打破 I/O 瓶颈：逐层传输与请求级异步

本节要回答的问题： 外部 KV Cache 传输延迟往往达到毫秒级。如何把传输时间藏进计算时间，让 GPU 尽可能不停下来？

vLLM KV Connector 先后落地了两种范式：**逐层传输 (Layer-wise Transfer)** 与 **请求级异步 (Request-level Async)**。两者在"对调度器的侵入程度"和"能隐藏的传输量"之间形成明显取舍。

3.1 范式一：逐层传输 - 对调度器完全透明

核心思路：不必等所有层的 KV Cache 全部到位才开始计算，每传完一层就立即执行该层的前向推理，同时后台继续搬运下一层。

在此模式下，Scheduler 无需任何修改 - 它像往常一样下发请求，KV Connector 在 Worker 内部自行完成"传一层、算一层"的流水线。这是 v1 KV Connector 最早内置的方案。

边界条件： 流水线只在"单层传输时间 $\leq$ 单层计算时间"时才能完全隐藏延迟。一旦网络带宽不足导致单层传输耗时超过单层计算耗时，GPU 仍会在该层计算结束后等待数据就绪。

维度	逐层传输
调度器改动	无
传输粒度	每层 KV
隐藏延迟上限	受限于最慢一层的传输耗时
显存占用特征	逐层释放/写入，无额外预留
适用场景	高带宽互联（NVLink、RDMA 等）或 KV 较小的短序列

当网络较慢时，逐层方案无法根治空转 - 需要把重叠粒度从层级提升到请求级。

3.2 范式二：请求级异步 - 加载期间先跑别的请求

请求级异步把视角从"同一请求内部的层间流水线"拉高到"不同请求之间的时间片交叠"。下图展示了这一范式的时序逻辑 - 关键在于理解"异步挂起"与"就绪恢复"两个信号如何衔接调度器和 Worker。

PARADIGM 2 · REQUEST-LEVEL ASYNC

范式二：加载期间,先跑别的请求



该机制由RedHat 工程师在v1 KV Connector 上后续贡献落地

图注：Request-level Async 的时序与三步信号流程。来源：演讲 PPT 第 11 页

图中的时序可以拆解为三个步骤：

- 1 **返回异步加载信号** - KV Connector 收到某请求（图中 Request A）的加载需求后，通过 `get_num_new_matched_tokens()` 向调度器返回一个异步标记：“数据还没到，但已在搬运”。
- 2 **预留显存，挂起请求** - Scheduler 为该请求提前分配 GPU blocks 用于后续写入，但不让 GPU 等待这些块被填满。GPU 转去执行其他已就绪的请求（图中 B、C）。
- 3 **加载完成，下轮调度** - Worker 后台写入完成后通知 Scheduler，该请求在下一轮调度中参与正常计算。

最小场景： batch 包含 A、B、C 三个请求，A 需从远端拉取 KV Cache。无异步时，GPU 等 A 的 KV 全部到位 → 计算 A → 计算 B → C，传输时间完全暴露。有请求级异步时，A 返回异步信号，GPU 先计算 B 和 C，与此同时 A 的 KV 在后台搬运。若 B、C 计算结束时 A 也加载完毕，下一轮直接开始 - 传输延迟被 B、C 的计算时间吸收。

3.3 代价与边界

请求级异步显著提升了 GPU 利用率，但伴随两个不可忽视的代价：

代价	具体表现
调度器复杂度飙升	Scheduler 必须理解“请求正在异步加载”这一新状态，维护就绪/未就绪队列，处理加载失败回滚等边界情况
显存长期占用	预留的 GPU blocks 在整个传输期间被占住却不产出推理结果；慢速网络或并发异步请求过多时，可用显存迅速收紧

该机制由 RedHat 工程师在 v1 KV Connector 上后续贡献落地。当外部链路延迟进一步增大，“占着显存等传输”的模式会成为新的资源瓶颈，系统需要更精细的预取策略来解耦显存分配与数据搬运。

四、显存保卫战：L2 预取机制的精妙实现

本节要回答的问题：当 KV Cache 位于远端且网络延迟高达数十毫秒时，如何让慢速传输彻底不占用 GPU 显存？

4.1 两阶段搬运 + 调度欺骗

L2 Prefetching（L2 预取）的核心策略拆成两步：

阶段	动作	GPU 显存开销
1) 后台预取	远端数据搬运到 CPU 内存 的 prefetch 缓冲区	零
2) 就绪加载	缓冲数据从 CPU 拷贝到 GPU 显存 的 paged KV cache	此时才分配 GPU blocks

关键在于：阶段 1) 期间，Scheduler 完全不知道该请求需要显存。实现这一效果的手段是让匹配函数 `get_num_new_matched_tokens()` 在数据尚未就绪时返回 `None`。

4.2 架构与数据流

下图展示了 L2 Prefetching 的三层数据流 - 理解"为什么要多加一层 CPU 缓冲"以及"返回 None 的实际效果"是本节重点。

PARADIGM 3 · L2 PREFETCHING

范式三: KV 还远时,先预取、不占显存



思路源自Dynamo 团队, 首先落地于LMCache

图注：L2 Prefetching 的整体架构与实现技巧。来源：演讲 PPT 第 12 页

图中三个层级：

- 最右侧：远端存储 / 磁盘（L2） - KV Cache 的"远处"，网络延迟不可忽视。

- **中间层：CPU 内存 prefetch 缓冲** - 承担中转角色，容量充裕且不消耗 GPU 显存配额。
- **最左侧：GPU 显存 paged KV cache** - 最终计算所需的位置，也是最稀缺的资源。

箭头 1) 标注为"后台进行，不占显存"，由异步 I/O 完成，对 Scheduler 透明。箭头 2) 标注为"就绪后加载，此时才分配 GPU blocks" - 只有 CPU 缓冲中的数据完整就绪，框架才向 Scheduler 报告匹配 token 数量，触发资源分配。

4.3 匹配函数返回 None 的因果链

- 1 **请求到达**：Scheduler 对每个待调度请求调用 `get_num_new_matched_tokens()`。
- 2 **预取未完成**：函数检测到 CPU prefetch 缓冲尚未就绪，返回 `None`。
- 3 **Scheduler 跳过**：`None` 的语义等价于"先别调度我"。Scheduler 不为该请求分配任何 GPU blocks，直接处理队列中的其他请求。
- 4 **后台传输持续**：远端数据正在向 CPU 内存搬运，不干扰 GPU 侧操作。
- 5 **预取完成**：CPU 缓冲就绪后，下一轮调度时函数返回实际匹配的 token 数。
- 6 **资源分配与加载**：Scheduler 正常分配 GPU blocks，CPU → GPU 拷贝完成后请求进入计算流水线。

显存占用时间被压缩到了"CPU→GPU

拷贝

+

计算"这一段，完全跳过了耗时最长的"远端→CPU"阶段。

4.4 最小状态演进

系统中有请求 A（正在 decode）和请求 B（需从远端加载 KV Cache）：

调度轮次	请求 B 匹配函数返回值	Scheduler 对 B 的处理	GPU 显存分配给 B?
t=1	<code>None</code> （预取刚开始）	跳过	否
t=2	<code>None</code> （预取进行中）	跳过	否
t=3	512（预取完成）	正常调度，分配 blocks	是

在 t=1 和 t=2 两轮中，原本会被 B 锁定的显存块全部留给了活跃请求。

4.5 收益与适用边界

收益：慢速传输期间显存零占用，GPU 显存全部服务于正在计算的请求。

边界条件：

- 该范式假设 CPU 内存有足够空间充当 prefetch 缓冲。大量请求同时预取时，CPU 内存可能成为新瓶颈 - 演讲材料未给出缓冲区容量管理策略细节。
- 返回 `None` 意味着请求的首次响应延迟（TTFT, Time To First Token）会增加一个完整的预取周期。对延迟敏感的在线场景需权衡命中率与预取开销。
- 该思路源自 Dynamo 团队的设计理念，首先在 LMCache 中落地实现。

有了完善的异步传输接口，vLLM

具备了接入强大外部缓存系统的能力。LMCache

正是首个深度整合的独立管理层。

五、走向独立守护进程：LMCache 的零竞争架构

本节要回答的问题：当 KV Cache 管理组件与推理引擎运行在同一 Python 进程里，GIL 竞争和视野受限是两个绕不开的结构性问题。LMCache 如何摆脱这些瓶颈？

5.1 Library 模式的结构性瓶颈

多数 KV Cache 管理库采用 **Library 模式**：将自身加载为 serving engine 进程内的一个模块。部署简单，但带来三层递进问题：

维度	Library 模式表现	根因
GIL (Global Interpreter Lock, 全局解释器锁) 竞争	KV 搬运的 Python 调度代码与 model forward 争抢同一把 GIL	同一进程，单 GIL
视野受限	每个库实例只能看到宿主进程持有的 GPU block	进程隔离
不可独立管理	无法对 KV 层单独做监控、升级或重启	生命周期绑定宿主

即使 KV 搬运本身在 GPU 上异步执行，Python 层面的元数据操作和回调仍需获取 GIL。并发量上升时，这些微小阻塞会累积为可观测的尾延迟抖动。同一台机器部署两个 vLLM 实例时，两个 KV Library 各自为政，无法共享缓存条目，造成显存浪费和命中率下降。

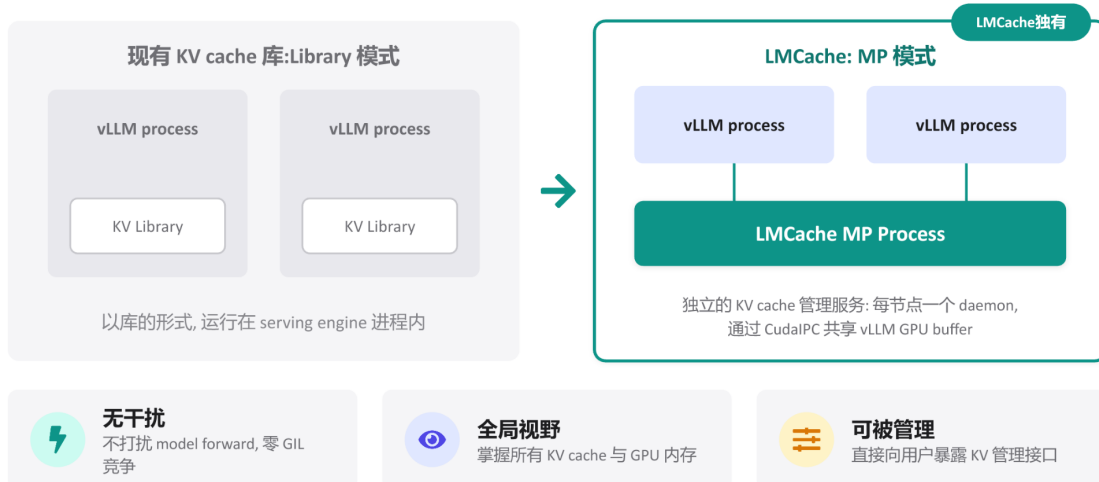
5.2 MP 守护进程架构

LMCache 摒弃了进程内模式，采用 **MP 模式 (Multi-Process 模式)** - 将 KV Cache 管理逻辑抽离为每节点一个独立的守护进程 (daemon)，通过 **CudaIPC** (CUDA Inter-Process Communication, CUDA 进程间通信) 与 vLLM 共享 GPU buffer。

下图将两种架构并排呈现，重点关注右侧 LMCache daemon 如何同时连接多个 vLLM 实例。

LMCACHE · ARCHITECTURE

LMCache 独特架构: KV Cache 的守护进程



图注: 左侧为传统 Library 模式, KV Library 嵌入 vLLM 进程内部; 右侧为 LMCACHE MP 模式, 独立守护进程通过 CudaIPC 连接多个 vLLM 实例。来源: 演讲 PPT 第 16 页

左侧每个 vLLM process 内部各包含一个 KV Library 方块 - 彼此不可见。右侧的关键变化是: **LMCache MP Process** 被提取到所有 vLLM 进程的外部, 以 daemon 形式运行, 通过连线与多个 vLLM process 相连, 连线代表 CudaIPC。

5.3 协作流程

- 1 **vLLM 发起请求** - Scheduler 决定某请求需要读写 KV Cache 时, vLLM 进程通过轻量 RPC 指令通知 LMCACHE daemon, 指令中包含目标 block 的 token hash 及 GPU buffer 地址。
- 2 **CudaIPC 映射** - daemon 利用 CudaIPC 直接获得 vLLM 所持 GPU buffer 的映射句柄, 在自己的进程上下文中读写该显存区域, 不经过 CPU 侧内存中转。
- 3 **零拷贝搬运** - daemon 在自己的 CUDA stream 上发起传输。由于操作完全在 daemon 进程内完成, vLLM 进程的 GIL 不会被触碰。
- 4 **完成通知** - 搬运结束后, daemon 通过 RPC 回调告知 vLLM, 后者在下一次调度循环中更新 block 元数据。

vLLM 进程只承担"发指令"和"收通知"两个极轻量操作, KV 数据的实际移动全部卸载给 daemon。

5.4 三项架构收益

- **无干扰** - daemon 拥有独立的 Python 解释器和 GIL, 即使执行复杂缓存淘汰策略, 也不与 model forward 产生锁竞争。
- **全局视野** - 单个 daemon 连接同节点所有 vLLM 实例, 能建立统一的 block 映射表。第二个 vLLM 实例请求相同 prefix 时, daemon 可通过 CudaIPC 直接从第一个实例的 buffer 读取, 无需重新计算。

- **可被管理** - daemon
以独立进程身份存在，可暴露缓存预热、手动淘汰、统计查询等专属管理接口。

5.5 最小场景

同一节点运行 vLLM 实例 A 和 B，共享一个 LMCache daemon：

- 1 实例 A 处理一条长 system prompt，prefill 完成后 daemon 记录该 prefix 对应的 GPU block 地址。
- 2 实例 B 收到相同 system prompt 的新请求，Scheduler 向 daemon 查询，发现 A 的 GPU buffer 中已有匹配 KV。
- 3 daemon 通过 CudaIPC 从 A 的显存映射读取，写入 B 的目标 block。不涉及 CPU 内存拷贝，A 和 B 的 model forward 均不被阻塞。

5.6 边界

CudaIPC 要求参与通信的 GPU 位于同一物理节点（或至少同一 PCIe/NVLink 域）。当 KV Cache 需要跨节点流转时，本地 CudaIPC 路径不再适用，需要借助 RDMA 等网络传输手段。此外，演讲材料未给出 LMCache daemon 在高并发场景下的具体吞吐或延迟基准数据，实际性能表现有待独立测评验证。

六、全局池化与硬件加速：Mooncake 的 RDMA 零拷贝实践

本节要回答的问题： 在跨节点 PD 分离架构中，如何实现极致的 KV Cache 传输性能？

6.1 跨节点传输的瓶颈

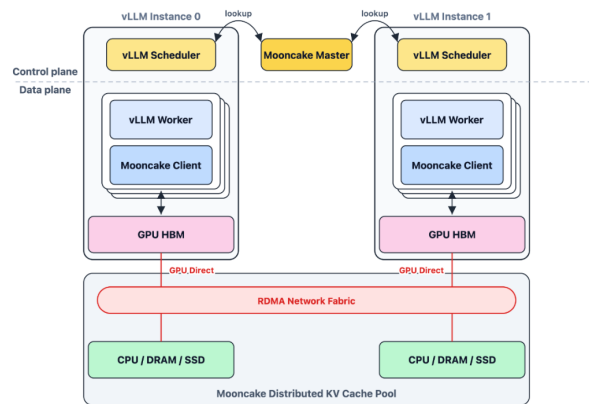
PD 分离架构下，Prefill 节点完成计算后需将 KV Cache 传递给 Decode 节点。传统路径中，数据必须先从 GPU 显存拷贝到主机内存（CPU DRAM），由 CPU 发起网络传输，对端收到后再从主机内存拷入 GPU。每次跨节点搬运至少经历**两次 PCIe 拷贝 + 一次网络传输**，CPU 全程参与。对于动辄数百 MB 的 KV Cache，这条路径很快成为吞吐瓶颈。

Mooncake 给出的答案是：将 GPU 显存直接注册为 RDMA (Remote Direct Memory Access, 远程直接内存访问) 缓冲区，借助 GPUDirect RDMA 在硬件层面完成端到端零拷贝传输。

6.2 MooncakeStoreConnector 的两层分工

下图展示了 MooncakeStoreConnector 的系统架构，核心在于理解调度侧和 Worker 侧如何分别与 Mooncake 基础设施交互，以及数据平面那条绕过 CPU 的直连箭头。

MooncakeStoreConnector



```

vllm
├── cude_utils
├── device_allocator
├── distributed
│   ├── __pycache__
│   ├── device_communicators
│   ├── ec_transfer
│   ├── elastic_ep
│   ├── ep1b
│   ├── kv_transfer
│   └── kv_connector
│       ├── v1
│       ├── hf3fs
│       ├── lmcache_integration
│       └── mooncake
│           └── store
│               ├── __init__.py
│               ├── connector.py
│               ├── coordinator.py
│               ├── data.py
│               ├── metrics.py
│               ├── protocol.py
│               ├── scheduler.py
│               └── worker.py
├── __init__.py
├── mooncake_connector.py
├── mooncake_utils.py
├── rdma_utils.py
└── stats.py
```

调度侧：vLLM 对新请求提示词分块哈希，向 Mooncake 主节点查询匹配的 KV 缓存，指导调度决策。
Worker 侧：各 GPU worker 嵌入 Mooncake 客户端，后台线程异步传输数据；KV 缓存注册为 RDMA 缓冲区，通过 GPUDirect RDMA 直接读写，无需 SM 参与或 CPU 中转

图注：MooncakeStoreConnector 的系统架构与数据平面。来源：演讲 PPT 第 31 页

图中两条独立路径：

层面	角色	核心动作	交互对象
调度侧	决策者	对提示词分块哈希，向 Mooncake Master 查询 KV Cache 命中	Mooncake Master (元数据节点)
Worker 侧	执行者	嵌入 Mooncake 客户端，后台线程异步完成实际传输	分布式 KV Cache 池及其他 GPU 节点

数据平面箭头直接连接 GPU HBM 和 Mooncake Distributed KV Cache Pool，标注为 "GPUDirect over RDMA Network Fabric" - 这条路径绕过了 CPU。

6.3 GPUDirect RDMA 的三步因果链

RDMA 允许一台机器的网卡直接读写另一台机器的内存，不经过任何一方的 CPU。GPUDirect RDMA 在此基础上更进一步：网卡可以直接访问 GPU 显存，省去 GPU → 主机内存的中间拷贝。

MooncakeStoreConnector 在 Worker 侧的做法：

- 1 **注册阶段：**vLLM Worker 启动后，将本地 KV Cache 所在的 GPU 显存区域注册为 RDMA 缓冲区（RDMA buffer），告知网卡和操作系统"这片显存可被远端直接读写"。
- 2 **传输阶段：**Mooncake 的 Transfer Engine (TE) 以后台线程运行，通过 RDMA 动词（verb）直接在两端 GPU HBM 之间建立数据通道。**无需 SM 参与或 CPU 中转。**
- 3 **完成确认：**RDMA 操作完成后，网卡通过完成队列（Completion Queue）通知 Mooncake 客户端，后者再通知 vLLM Worker 该 KV Cache 已就绪。

传输速度上限不再取决于 CPU 吞吐或 PCIe 拷贝次数，而是**直接受限于底层 RDMA 网络带宽**。

6.4 最小例子：跨节点 Prefill→Decode 搬运

节点 A 专职 Prefill，节点 B 专职 Decode，两者通过 RDMA 网络互联。

第一步：调度侧查询。 节点 B 的 Scheduler 对提示词分块计算哈希值，向 Mooncake Master 查询：这些 KV 块是否已存在于全局缓存池中？若命中，得知数据位于节点 A 的 GPU 0 上。

第二步：Worker 侧传输。 节点 B 的 Worker 中嵌入的 Mooncake 客户端发起 RDMA Read：



在这条路径中：节点 A 的 CPU 不参与搬运；节点 B 的 CPU 不参与搬运；双方 GPU SM 不被占用，可继续执行其他推理计算。

第三步：就绪通知。 RDMA 传输完成后，节点 B 的 Worker 收到通知，Decode 阶段立即使用到位的 KV Cache，无需额外显存内拷贝。

6.5 性能优势与边界

环节	传统路径	Mooncake GPUDirect RDMA
GPU→主机内存拷贝	需要	省去
CPU 参与网络发送	需要	省去
主机内存→GPU 拷贝	需要	省去
SM 占用	可能	无
传输带宽上限	受 PCIe + CPU 调度限制	受 RDMA 网络带宽限制

边界条件：

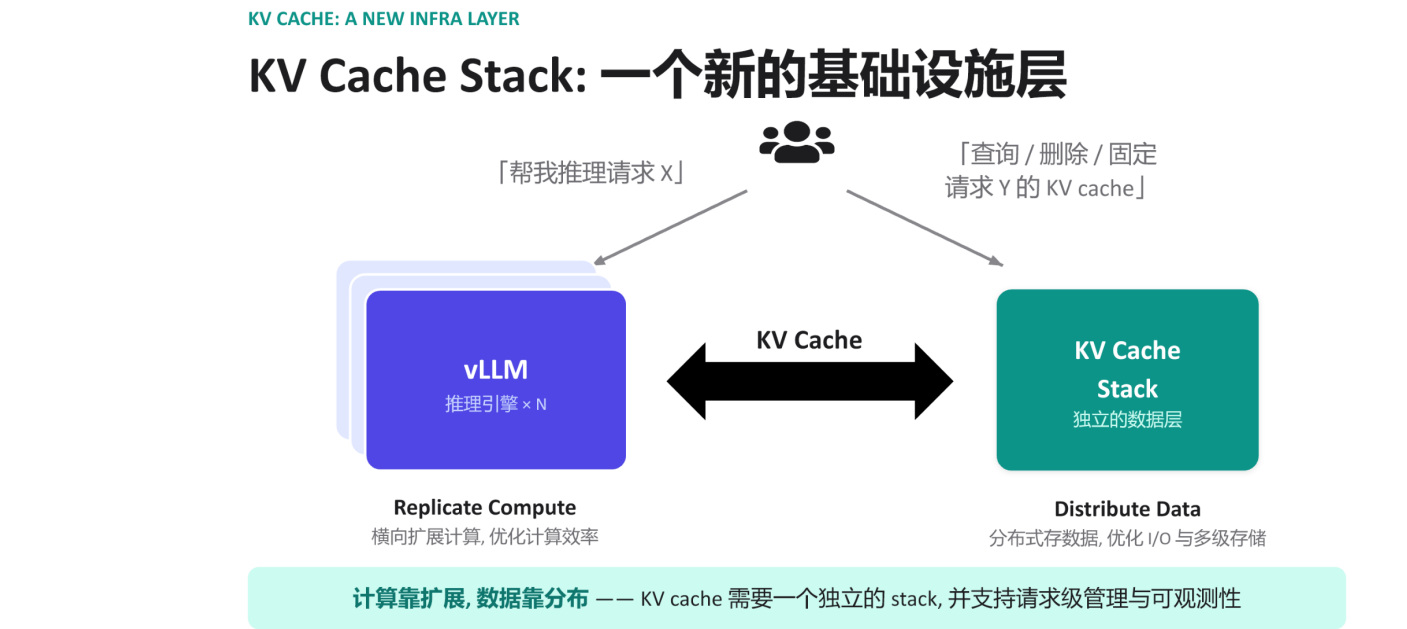
- GPUDirect RDMA 要求网卡和 GPU 均支持该特性，且两者最好位于同一 PCIe 交换域以获得最佳性能。并非所有部署环境满足此条件。
- 演讲材料未给出具体的带宽数字或延迟基准测试数据，实际传输速率取决于网络拓扑、网卡型号和 RDMA 协议类型 (InfiniBand vs. RoCE) 。
- Mooncake 的分布式 KV Cache 池支持 CPU/DRAM/SSD 多级存储。当 KV Cache 从非 GPU 层级加载时，是否仍能维持完整的零拷贝语义，需结合具体部署验证。

七、KV Cache Stack 的崛起与工程边界

本节核心问题： 当 KV Cache 从推理引擎的内部状态演变为独立基础设施层，系统获得了什么？在真实部署中又会碰到哪些物理天花板与工程代价？

7.1 范式转变：计算与数据的正交扩展

纵观前文所有机制 - Connector 接口、异步传输范式、全局零拷贝池化 -
它们最终指向同一个架构愿景：将 KV Cache 视为一个可独立演进的数据层。
下图把这一愿景具象化为两个正交扩展的平面。



图注：KV Cache Stack 概念架构。来源：演讲 PPT 第 19 页

维度	左侧：推理引擎集群	右侧：KV Cache Stack
扩展策略	Replicate Compute - 横向复制实例	Distribute Data - 分布式存储与多级缓存
优化目标	计算吞吐与 GPU 利用率	I/O 带宽与存储命中率
请求类型	"帮我推理请求 X"	"查询 / 删除 / 固定请求 Y 的 KV Cache"
运维粒度	实例级弹性伸缩	请求级管理与可观测性

两侧通过双向箭头连接。推理引擎只需关心"发起推理"与"声明 KV Cache 层。这正是 PD 分离得以落地的底层前提：Prefill 节点产出 KV Cache 写入 Stack, Decode 节点从 Stack 拉取，二者无需共享 GPU 显存甚至物理机器。

因果链：KV Cache 独立成层 → 计算节点无状态化 → 引擎实例可按需水平扩展 → Prefill 与 Decode 各自配置最优硬件比例 → 整体推理成本下降。

7.2 物理约束：带宽决定上限

零拷贝和异步传输能消除侧的多余开销，但最终吞吐被网络硬件拓扑锁死。需要注意的边界情形：

CPU

- **非 RDMA 网络退化：** 在普通 TCP/IP 链路上，零拷贝语义仍然可用，但实际带宽可能降低一个数量级，PD 分离的时延收益会被传输瓶颈抵消。
 - **多级存储冷热不均：** KV Cache Stack 支持 GPU 显存、主机内存、远端存储等多级层次。热点请求集中在低速层时，统计上的缓存命中率优势并不能掩盖单次 miss 带来的长尾延迟。
- 演讲材料未给出具体的网络带宽阈值或退化曲线数据，部署时应针对实际拓扑做基准测试。

7.3 工程代价：异步复杂度的蔓延

KV Cache 生命周期从引擎内部剥离后，Scheduler 需要额外维护的状态显著增加：

- 1 **传输状态追踪** - 每个请求的 KV Cache 可能处于"发送中 / 已到达 / 校验失败 / 已固定"等多种中间态，Scheduler 必须为每种状态设计回退路径。
- 2 **请求级管理** - Stack 层要支持"查询、删除、固定"三类操作，Scheduler 不仅要编排 GPU 算力，还要编排分布式缓存的引用计数与淘汰策略。
- 3 **可观测性成本** - 独立数据层需要自己的监控指标（命中率、传输延迟、存储水位），与推理引擎的指标体系合并后，运维维度至少翻倍。

这三项开销在单机部署中几乎不可感知，但在数十乃至上百个引擎实例的集群里，会成为系统可靠性的主要挑战点。

7.4 部署评估建议

评估维度	关注指标	不满足时的退路
网络带宽	节点间单向带宽是否足以在一个 Decode step 内完成 KV 传输	回退至同机 PD 共置，放弃分离收益
调度复杂度	Scheduler 异步回调链深度与异常恢复覆盖率	降级为同步阻塞传输，牺牲吞吐换稳定
多级存储	热层（GPU 显存）容量能否覆盖活跃请求集	缩短最大序列长度或增大显存配比

结论与局限

核心结论：

- 1 **KV Cache 正在从推理引擎的内部组件演变为独立的数据基础设施层。**
这一转变使得计算与数据可以按各自最优策略独立扩展。
- 2 **调度与执行的解耦是全部后续优化的架构前提。** vLLM v1 通过六个注入式接口将"何时加载"与"如何搬运"彻底分离，为异步传输和外部存储接入提供了稳定的抽象。

- 3 **三种异步传输范式逐级递进地解决了 GPU 空转问题。**
逐层传输零侵入但受限于单层带宽；请求级异步大幅提升利用率但占用显存；L2 预取通过 CPU 缓冲彻底解耦显存分配与慢速传输。
- 4 **LMCache 的 MP 守护进程模式消除了 GIL 竞争并获得节点级全局视野。** CudaIPC 使得跨 vLLM 实例的零拷贝共享成为可能。
- 5 **Mooncake 通过 GPUDirect RDMA 将跨节点 KV Cache 传输压缩为一次硬件级端到端搬运。** PD 分离架构中 KV Cache 传输的性能上限仅取决于 RDMA 网络带宽。
- 6 **极致的传输性能高度依赖底层硬件拓扑的支持。** GPUDirect RDMA 要求网卡与 GPU 均具备对应能力，非理想网络环境下性能可能显著退化。
- 7 **异步机制在提升 GPU 利用率的同时，带来了调度器复杂度的非线性增长。**
传输状态追踪、请求级缓存管理和跨层可观测性共同构成了工程上的主要维护成本。

明确局限：

- 演讲材料未提供各异步范式的定量性能对比数据，本文分析基于架构层面的定性推演。
- LMCache 和 Mooncake 的零拷贝传输效果严格受限于底层硬件配置（如 RDMA 网卡型号、PCIe 拓扑），在非理想环境下可能无法达到预期性能。
- 请求级异步在慢速网络下会导致 GPU 显存长时间被占用，实际部署时需结合系统压力谨慎评估。
- 在网络条件或运维能力不达标的环境下，保守的同机共置部署仍可能是更务实的选择。